Microcontroladores - Vol. 1

Gabriel Duarte Marques

Março 2025

Sumário

1 Introdução aos Microcontroladores **5**
1.1 O que são Microcontroladores 5
1.2 Microcontroladores versus Microprocessadores 6
1.3 Componentes Básicos de um Microcontrolador 6
1.4 Tipos de Microcontroladores 6
1.4.1 Classificação por Tamanho da Palavra 7
1.4.2 Plataformas Alternativas 7
1.4.3 Classificação por Família/Fabricante 7
1.4.4 Classificação por Arquitetura 8

2 Arquitetura dos Microcontroladores **9**
2.1 Arquiteturas Fundamentais 9
2.1.1 Arquitetura Harvard 9
2.1.2 Arquitetura von Neumann 10
2.1.3 Comparação entre Harvard e Von Neumann 11
2.2 Componentes Básicos da Arquitetura 11

3 PICs: Fundamentos Teóricos **13**
3.1 Visão Geral da Família PIC 13
3.1.1 Relações Intra-Familiares 13
3.1.2 Formatos Físicos ou Encapsulamento 13
3.1.3 Famílias Principais 14
3.2 Estrutura do PIC16F628A 15
3.2.1 Portas e Pinagem (pinout) 15
3.3 Organização de Memória 16
3.3.1 Memórias 16
3.3.2 Principais Registradores 16

4 Configurações de IDE e Projeto **17**
4.1 Ferramentas Necessárias 17
4.2 Criação de Projeto 17

4.2.1 Bits de Configuração do PIC 19
4.2.2 Configurações de Clock 21

5 Programação Prática de PIC **23**
5.1 GPIO e Gerenciamento de Portas 23
5.1.1 Escrita do Código 24
5.1.2 Montagem de Circuito 24
5.2 Gravação do PIC 26
5.2.1 Ferramentas Necessárias 26
5.2.2 Processo de Gravação com Soquete ZIF 26
5.3 Montagem do Cristal 28

6 Módulos do PIC16 **29**
6.1 Timers e Interrupções 29
6.1.1 Timers 29
6.1.2 Configuração de Prescaler/Postscaler 30
6.1.3 Interrupções 30
6.2 Comparadores 33
6.3 Comunicação Serial (USART) 34

7 Programação Avançada de PIC **35**
7.1 Option Registers 35
7.2 Status 35
7.3 Bit Baggin 35
7.4 Debounce 35
7.5 Otimização 35

8 Periféricos Essenciais **37**
8.1 LCDs 37
8.2 Conversor A/D 37

9 Projetos Baseados em Problemas **39**
9.0.1 Sistema de Controle 39
9.0.2 Monitor de Temperatura 39
9.0.3 Interface Homem-Máquina 39

Capítulo 1

Introdução aos Microcontroladores

1.1 O que são Microcontroladores

Nesse momento é pressuposto que em algum momento você já ouviu ou utilizou o Arduíno, a famosa placa azul é geralmente a porta de entrada para o mundo da eletrônica e por exemplo no modelo UNO da marca temos soldado na placa um microcontrolador de modelo ATmega328P e além dele existem outros componentes que fazem o Arduíno funcionar e facilitar a montagem de circuitos.

Assim como o microcontrolador usado no Arduíno temos de outras fabricantes seus próprios modelos de microcontroladores a Microchip tem os PIC e suas respectivas famílias e a Intel tem os 8051 e famílias, cada fabricante, cada modelo, tem suas características por isso se deve ao utilizar qualquer componente eletrônico que seja se faz necessário ao menos ler por cima o **datasheet** que é um documento criado pelo fabricante que resume o desempenho e outras características técnicas de um produto e que fornece detalhes suficientes para que possa ser usado.

Definido como se parecem e de onde vieram, os **Microcontroladores** são pequenos computadores que estão por toda a parte no nosso cotidiano e note a seguir as nuâncias de **Controle** dele que vão desde os objetos mais simples como micro-ondas, ar-condicionados, controles remotos aos mais complexos como carros, aviões, equipamentos médicos e entre outros pois as implementações são infinitas, principalmente com o advento das IA's, diferente de um computador pessoal que se precisa de várias peças para funcionar, como processador, placa-mãe, disco rígido, fonte e outros, os microcontroladores contém todas esses componentes em um único chip e tendo um consumo energético menor se comparado ao seu irmão maior.

Com base na descrição e nos exemplos anteriores podemos então inferir que a palavra "Micro"pode significar pequeno e "Controlador"pode significar controle de processos então o conjunto **MicroControlador** pode significar pequeno controlador de processos.

1.2 Microcontroladores versus Microprocessadores

Os Microcontroladores e Microprocessadores antes da década de 90 já existiam, mas foi durante essa época que com o avanço da miniaturização, o avanço das técnicas de processamento de silício e a fabricação de chips resultaram na capacidade de colocar cada vez mais transistores em um único núcleo, foram ganhando cada vez mais espaço e começaram a ser amplamente adotados em bens de consumo e sistemas de controle. Um microprocessador é essencialmente um processador de um computador em escala de menor tamanho, logo, ele necessita de todos os componentes que um computador também precisa para o seu funcionamento.

Os microcontroladores por outro lado contém tudo que se necessita para o controle de sistemas em um único **CI** (Circuito Integrado) com memórias embutidas, conversores e entre outros componentes.

1.3 Componentes Básicos de um Microcontrolador

- **Unidade central de processamento (CPU):** é o "cérebro"do computador, a CPU serve como o responsável pela execução de instruções e controle de operações.
- **Memória:** os microcontroladores contêm memória volátil a chamada **RAM** que armazena dados temporários para execução e memória flash não volátil (um tipo avançado de **EEPROM**) que armazena a memória de programa.
- **Periféricos:** um microcontrolador pode conter vários componentes auxiliares, como interfaces de entrada/saída (E/S) e isso incui uma infinidade de componentes aqui estão alguns deles temporizadores, contadores, conversores de sinal, módulos e entre outros.

1.4 Tipos de Microcontroladores

Os microcontroladores podem ser classificados de diversas formas, sendo as mais comuns baseadas em sua arquitetura, desempenho, tamanho da palavra e aplicação. Conhecer estas classificações é importante para a seleção adequada do dispositivo para cada projeto.

1.4.1 Classificação por Tamanho da Palavra

Os microcontroladores podem ser classificados de acordo com o tamanho de dados que processam:

- **8 bits:** São os mais comuns para aplicações de controle simples, como eletrodomésticos e brinquedos. Exemplos: AVR (Arduino), PIC16, 8051.
- **16 bits:** Oferecem maior poder de processamento para aplicações intermediárias. Exemplos: PIC24, MSP430.
- **32 bits:** Para aplicações que exigem maior capacidade de processamento e recursos. Exemplos: ARM Cortex-M, PIC32, ESP32.
- **64 bits:** Encontrados em aplicações embarcadas de alto desempenho.

1.4.2 Plataformas Alternativas

Além dos microcontroladores tradicionais, é importante mencionar os **FPGAs** (Field Programmable Gate Arrays) como uma plataforma alternativa para implementação de arquiteturas de microcontroladores. Embora os **FPGAs** não sejam microcontroladores, eles oferecem a flexibilidade de implementar arquiteturas personalizadas de microcontroladores em hardware reconfigurável.

Também temos os **CLPs (Controlador Lógico Programável):** Sistemas industriais modulares que incorporam microcontroladores em sua CPU. São projetados para ambientes industriais severos, resistência a vibrações, a poeira, a umidade e interferências eletromagnéticas, mas possuem custo elevado comparado a soluções baseadas em microcontroladores puros.

1.4.3 Classificação por Família/Fabricante

Existem diversos fabricantes no mercado, cada um com suas próprias famílias de microcontroladores:

- **Microchip:** Famílias PIC (PIC10, PIC12, PIC16, PIC18, PIC24, PIC32)
- **Atmel:** Famílias AVR (utilizados no Arduino), megaAVR, tinyAVR
- **Texas Instruments:** MSP430, C2000, Tiva
- **STMicroelectronics:** STM8, STM32 (baseados em ARM)
- **NXP:** S08, Kinetis, LPC (baseados em ARM)
- **Espressif:** ESP8266, ESP32 (populares para IoT)
- **Intel:** 8051 e suas derivações

1.4.4 Classificação por Arquitetura

Os microcontroladores podem ser classificados conforme a arquitetura de seu conjunto de instruções:

- **CISC (Complex Instruction Set Computer):** Utiliza um conjunto de instruções complexo e extenso, capaz de executar operações elaboradas com uma única instrução um exemplo de microcontroladores com arquitetura CISC é os da família 8051 da Intel.
- **RISC (Reduced Instruction Set Computer):** Emprega um conjunto reduzido de instruções simples e otimizadas um exemplo de microcontroladores com arquitetura RISC são os baseados em ARM.

Capítulo 2

Arquitetura dos Microcontroladores

2.1 Arquiteturas Fundamentais

2.1.1 Arquitetura Harvard

Desenvolvida originalmente na Universidade de Harvard em 1944, esse tipo de arquitetura é caracterizada pela separação física das memórias de programa e dados, com barramentos independentes para cada uma. Este tipo de projeto permite acesso simultâneo às duas memórias, aumentando a eficiência em aplicações de tempo real.

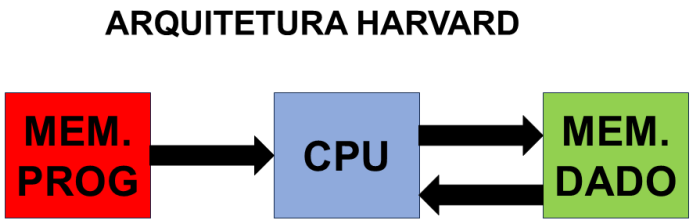


Figura 2.1: Diagrama da arquitetura Harvard

2.1.2 Arquitetura von Neumann

Proposta por John von Neumann em 1945, unifica programas e dados em um único barramento e espaço de memória, simplificando o projeto, mas essa arquitetura cria o **gargalo de von Neumann**, onde a CPU não pode buscar instruções e dados simultaneamente. Nos computadores modernos, o gargalo é mitigado por:

- **Caches L1 separadas:** Adoção de arquitetura *Harvard modificada* com L1i (instruções) e L1d (dados) para acesso paralelo.
- **Técnicas de paralelismo:** Pipelining, execução fora de ordem e multithreading.
- **Memória unificada superior:** Caches L2/L3 e RAM mantêm o modelo von Neumann.

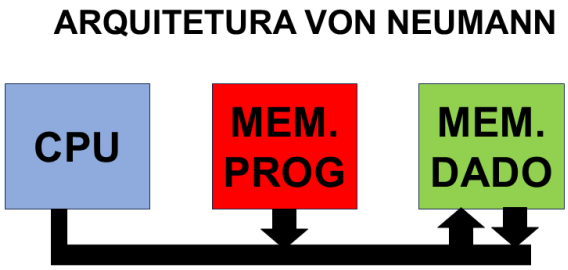


Figura 2.2: Diagrama da arquitetura Von Neumann

2.1.3 Comparação entre Harvard e Von Neumann

As arquiteturas Harvard e Von Neumann são fundamentais no design de microcontroladores e microprocessadores, cada uma com suas características distintas e aplicações específicas. Observando as figuras anteriores 2.1 e 2.2 é possível ver que para sair de uma arquitetura Harvard para uma de Von Neumann é necessário somente uma porta lógica **AND** que sirva de junção para as memórias de programa e de dados, agora o exercício contrário não é simples de se perceber.

A seguir uma tabela que exemplifica as diferenças entre as duas arquiteturas fundamentais:

Característica	Harvard	Von Neumann
Barramentos	Separados	Único
Segurança	Maior	Menor
Custo	Maior	Menor
Aplicação	Sistemas Embarcados	Computação geral

Tabela 2.1: Comparação entre as arquiteturas

2.2 Componentes Básicos da Arquitetura

A arquitetura básica de um microcontrolador é composta por componentes interconectados que trabalham em conjunto para executar tarefas de controle. A Figura 2.3 ilustra a organização típica desses componentes:

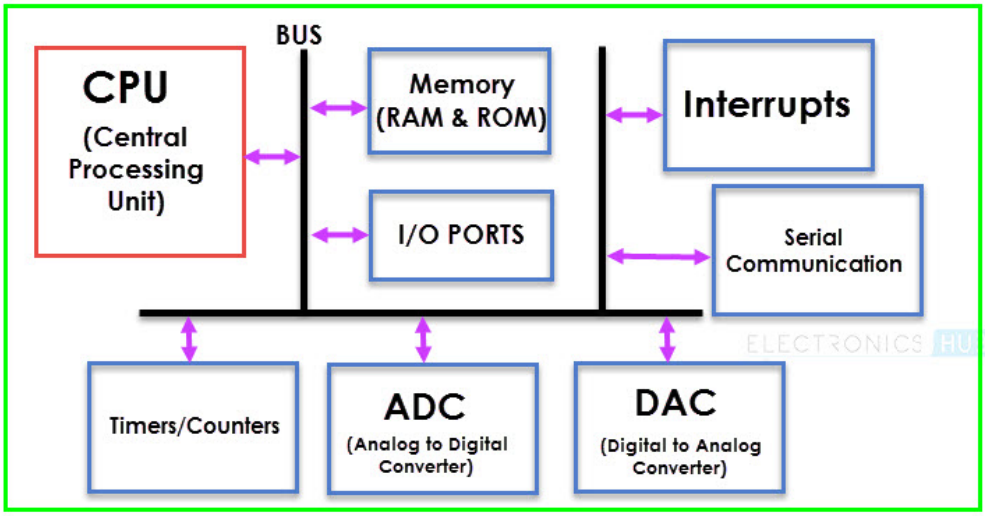


Figura 2.3: Arquitetura Básica

Os principais componentes e suas funções são:

- **CPU (Unidade Central de Processamento)**: Executa instruções armazenadas na memória de programa, controlando o fluxo de dados entre os componentes. Em microcontroladores RISC, opera com um conjunto reduzido de instruções otimizadas.
- **Memória de Programa (Flash/EEPROM)**: Armazena o código-fonte do firmware de forma não volátil. Em um Arduino Uno, por exemplo, possui 32 KB para armazenar instruções.
- **Memória de Dados (RAM)**: Armazena variáveis temporárias durante a execução do programa. Possui acesso rápido, mas é volátil (perde dados ao desligar).
- **Barramento de Sistema**: Conjunto de linhas de comunicação que interconectam CPU, memória e periféricos. Inclui:
  - Barramento de dados (transfere instruções/dados)
  - Barramento de endereços (localiza memória/periféricos)
  - Barramento de controle (sincroniza operações)
- **Periféricos**: Circuitos especializados para funções específicas:
  - Temporizadores/Contadores (ex: TIMER0 no PIC16F877A)
  - Conversores Analógico-Digital (ADC)
  - Módulos de comunicação (UART, SPI, I2C)
  - Portas GPIO (General Purpose Input/Output)
- **Interface de E/S**: Conecta o microcontrolador a sensores, atuadores e outros dispositivos externos.

Capítulo 3

PICs: Fundamentos Teóricos

3.1 Visão Geral da Família PIC

São microcontroladores de arquitetura PIC (Peripheral Interface Controller) da Microchip Technology e passou por uma evolução significativa desde seu surgimento em 1975. Sua trajetória reflete as demandas tecnológicas de cada era, com adaptações que mantêm compatibilidade retroativa em algumas famílias e rupturas em outras.

3.1.1 Relações Intra-Familiares

As famílias de microcontroladores PIC são projetadas com uma filosofia de *hardware modular*, onde modelos da mesma série compartilham características fundamentais, variando apenas em aspectos específicos como memória, portas e pequenas diferenças na programação para atender diferentes nichos de aplicação. Por exemplo, os PICs de modelo **16F628A** compartilham a mesma família que os modelos **16F627A**, **16F648A** e compartilham o mesmo ***datasheet***.

3.1.2 Formatos Físicos ou Encapsulamento

- **DIP/PTH (Dual In-line Package):**
  - Pinos paralelos em duas fileiras
  - 6 a 40 pinos - DIP40,DIP6...
  - Fácil soldagem
- **SMD (Surface-Mount Device):**
  - Montagem direta na superfície
  - Tipos comuns: SOIC, QFN, MLF
  - Compacta

– Uso de difíceis técnicas de soldagem

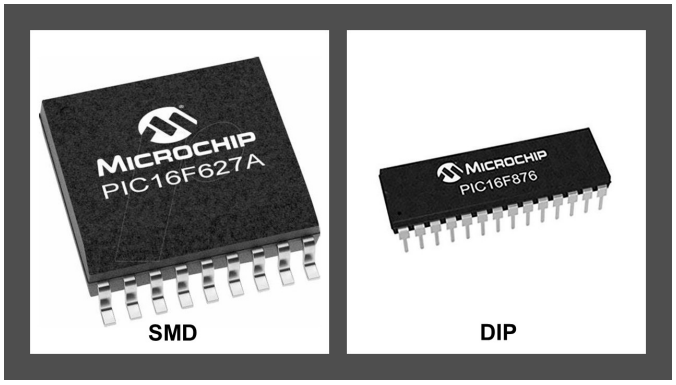


Figura 3.1: Diferença do Chip DIP e SMD

3.1.3 Famílias Principais

- **PIC10/12/16 (8-bit):** Compartilham várias semelhanças em sua arquitetura e conjunto de instruções, o que permite que o código seja facilmente portado entre si com pequenas modificações:

- **PIC18 (Enhanced 8-bit):** Tem um conjunto de instruções expandido,pilha e tamanho de bits maior com recursos de programação mais avançados.

- **PIC24/33 (16-bit):** Permitem operações mais complexas, pode ter interface gráfica integrada e oferece suporte nativo a protocolos **CAN** e **USB**.

- **PIC32 (32-bit):** Além da maior velocidade de processamento, oferrece suporte nativo a **Ethernet**, **USB** e **DMA** para programação profunda e comunicação.

3.2 Estrutura do PIC16F628A

3.2.1 Portas e Pinagem (pinout)

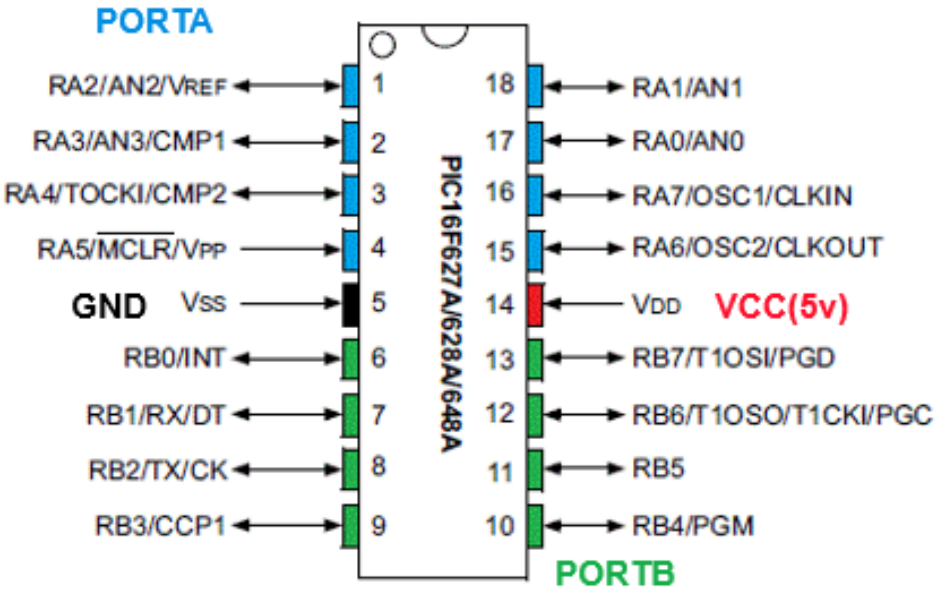


Figura 3.2: Pinagem do PIC16F628A

- **PORTA (Pinos Azuis):**
  - Pinos 1-4, 17-18 - Porta bidirecional de I/O
  - Capacidade de entrada analógica **(AN0-AN3)**
  - Funções especiais como comparadores, masterclear e entre outros.
  - Oscilador externo **(RA6-RA7)** nos pinos 16-17
- **PORTB (Pinos Verdes):**
  - Pinos 6-13 - Porta bidirecional de I/O digital
  - Comunicação serial **(TX-RX)**
  - Interrupção externa **(INT)**
  - Módulo CCP **(Capture/Compare/PWM)**
  - Gravação de programa **PGD, PGC e PGM**
- **Alimentação:**
  - VSS (Pino 5) - Terra **(GND)**
  - VDD (Pino 14) - Alimentação positiva **(5V)**

3.3 Organização de Memória

O microcontrolador PIC16F628A possui uma arquitetura de memória Harvard modificada, com separação física entre memória de programa e dados. Sua organização de memória é crucial para entender seu funcionamento e programação.

3.3.1 Memórias

- **Memória de Programa (Flash):** A memória de programa do PIC16F628A é do tipo Flash, permitindo reprogramação elétrica e é o local onde fica armazenado a progração do PIC.
- **Memória de Dados (RAM):** A memória de dados do PIC16F628A é organizada em bancos, banco 0 e banco 1, a RAM é classificada em dois tipos:
  - **SFR (Registradores de Funções Especiais):** Controlam o funcionamento do dispositivo.
  - **GPR (Registradores de Propósito Geral):** Para armazenamento de dados do temporários.
- **EEPROM de Dados:** É uma memória não-volátil presente no PIC16F628A que tem a função de armazenar dados permanentes que precisam ser mantidos mesmo quando o dispositivo é desligado.
- **Memória de Configuração:** É uma área especial da memória de programa (Flash) que define o comportamento fundamental do microcontrolador armazenando os bits de configuração.

3.3.2 Principais Registradores

Registrador	Função	Endereço
STATUS	Status da ALU e seleção de banco	0x03
TRIS	Direção dos pinos do PORTB	0x86
PORT	Estado dos pinos de I/O	0x06
INTCON	Controle de interrupções	0x0B

Tabela 3.1: Registradores Especiais por Banco

Capítulo 4

Configurações de IDE e Projeto

4.1 Ferramentas Necessárias

Para a começarmos a programar em PICs, precisaremos assim como no Arduino utilizar uma IDE compatível, nesse caso a fabricante tem seu próprio ambiente de desenvolvimento o MPLAB-X da Microchip e juntamente com ele precisaremos escolher um compilador para programar no caso vamos usar um de 8 bits padrão para que possamos programar em C e utilizaremos o compilador XC8 também da Microchip. Os links para download estão abaixo:

- MPLAB-X IDE: https://www.microchip.com/en-us/tools-resources/develop/mplab-x-ide
- XC8 C Compiler: https://www.microchip.com/en-us/tools-resources/develop/mplab-xc-compilers/downloads-documentation#XC8

Faça o download das ferramentas de acordo com o seu sistema operacional, depois disso instale conforme recomenda o fabricante. Ao iniciar o MPLAB X pela primeira vez, será solicitado que você selecione o diretório que será salvo os programas.

4.2 Criação de Projeto

O MPLAB X IDE organiza todo o desenvolvimento dentro de projetos, que agrupam códigos-fonte, arquivos de cabeçalho e configurações, iremos usar o modelo **PIC16F628A** para criar um novo projeto, siga os passos abaixo:

1. Abra o MPLAB X IDE.
2. Selecione **File** → **New Project** ou clique no ícone **Create New Project** na página inicial.
3. Na aba de novo projeto que aparece, selecione:

- Categories: **Microchip Embedded**
- Projects: **Application Project(s)**
- Clique em **Next**

4. Na aba de seleção de dispositivo, selecione:
   - Family: **All Families ou Escolha a família que deseja usar**
   - Device: **Escreva o modelo que deseja programar**
   - Tool: **Deixe em No tool ou se estiver com o PICkit conectado ao PC selecione o mesmo**
   - Clique em **Next**
5. Em Select Header **Não Mexa e clique em Next**
6. Na aba de Select Compiler, selecione o compilador
   - **XC8(versão)**
7. clique em **Next**
8. Na aba Select Project Name, coloque o nome do projeto
9. Clique em **Finish** para concluir a criação

Abaixo estão algumas imagens do processo:

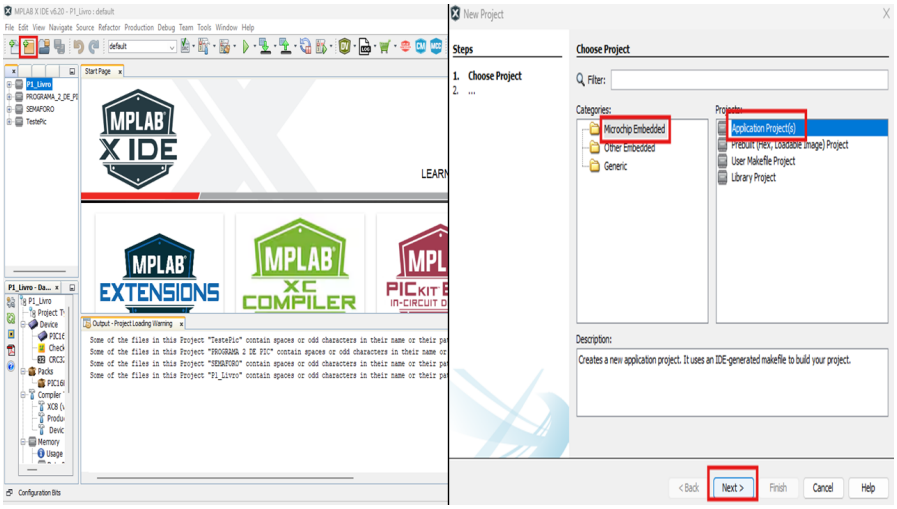


Figura 4.1: Tela de criação de projeto e dispositivo no MPLAB X IDE

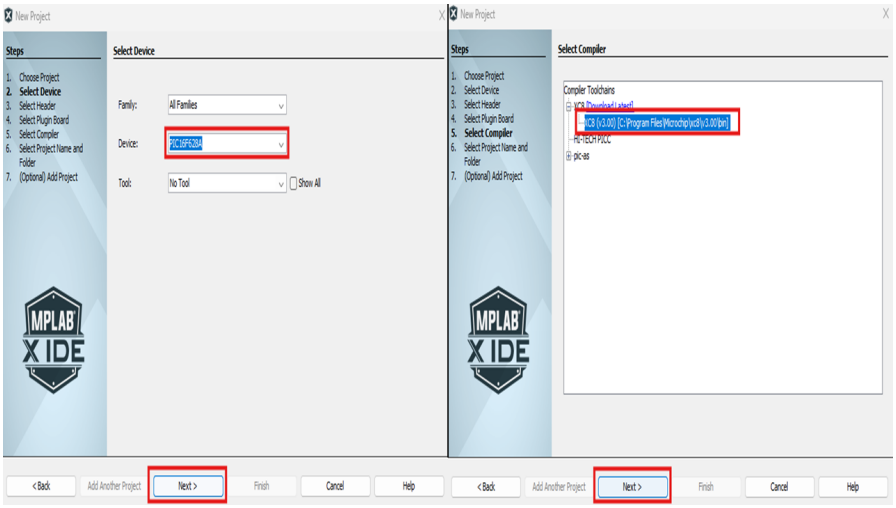


Figura 4.2: Seleção do microcontrolador PIC16F628A e compilador

Agora com o Projeto criado aba da lateral esquerda na qual tem o nome do projeto maximize-o, clique em **Source Files** e com o botão direito nele **New→ main.c** que é diferente de **C Main File** que é para outros compiladores diferentes do **XC8**.

4.2.1 Bits de Configuração do PIC

Com a **Main.c** feita, clique em **Production→Set Configuration Bits** e nessa etapa você irá configurar o PIC para receber o programa, os **Configuration Bits** variam entre cada Microcontrolador e após escolher as configurações clique em ***Generate Source Code to Output*** ele ir gerar uma série de **pragma configs** que você deve copiar e colar antes dos **includes** no código.Na aba de Configuration Bits ele da uma pequena explicação do que significa cada configuração, mas em 80 % as configurações são parecidas mudando um pouco o nome dela contudo mantendo o mesmo funcionamento.A seguir uma explicação de algumas delas:

- **FOSC(Frequency Oscillator):** Responsável pela fonte de frequência que o PIC irá utilizar para sincronizar todas as operações e leituras.
  - **XT:** Utiliza um Cristal externo de média frequência entre 4-20 MHz.
  - **HS:** Utiliza um Cristal externo de alta frequência maiores que 8 MHz.
  - **EXTRC:** Oscilador externo baseado em Resistor e Capacitor, mas com baixa precisão e estabilidade.
  - **INTOSC:** Utiliza um Oscilador interno se o μC tiver.
  - **LP:** Utiliza um Cristal externo de baixa frequência.
- **WDTE(Watch Dog Timer):** Uma medida de segurança que reinicia o PIC ao ser chamado via software ou de forma periódica.

  - **ON**
  - **OFF**
  - **NSLEEP:** Desligado no modo Sleep.
  - **SWTEN:** ON ou OFF de modo dinâmico via software.
- **PWRTE(Power-Up Timer):** Garante a estabilidade de tensão, atrasando a inicialização, recomendável para fontes de tensão instáveis.
  - **ON:** Verifica se tensão está estável.
  - **OFF**
- **BOREN (Brown-Out):** Reinicia o μC se a tensão estiver abaixo de um limite programado.
  - **ON:** Se ligado junto com **PWRTE** ele espera a estabilidade de tensão primeiro.
  - **OFF**
- **LVP(Low Voltage Programming):** Habilita o μC a programar em tensão de **3.3v** (requer um circuito específico).
  - **ON**
  - **OFF**
- **CPD(Code Protection Data):** Protege a leitura e cópia dos dados armazenados na **EEPROM**.
- **CP(Code Protection):** Protege a leitura dos dados armazenados na **Memória Flash**.
  CPs podem ser ON ou OFF
- **WRT(Write Protection):** Protege os dados da **Memória Flash** evitando reescrita de dados após a inicialização do programa.
  - **256:** Últimos 256 bits
  - **Half:** Metade de baixo
  - **Upper Half:** Metade de cima
  - **Quarter ou 1/4:** 25 % do programa.

ATENÇÃO

- Ativar `CP`, `CPD` ou `WRT` bloqueia permanentemente:
  - Leitura do firmware (engenharia reversa)
  - Regravação do microcontrolador
- **Nunca** habilite estas opções durante o desenvolvimento
- Em produção, verifique 3x o código antes de ativar as proteções
- PICs protegidos **não podem ser reescritos e recuperados**

4.2.2 Configurações de Clock

Seguindo nas configurações básicas do PIC, no código depois dos **includes** se deve definir um valor para o ***Cristal***

- **Padrão**: Se nenhum `#pragma config FOSC` for feita, o PIC16F628A utiliza o oscilador interno de 4 MHz (`INTOSC`) por padrão.
- **Comando Universal**: O comando `#define _XTAL_FREQ` deve ser usado **sempre**, independentemente do tipo de oscilador e se não utilizado o valor padrão do Cristal será 4 MHz.

Capítulo 5

Programação Prática de PIC

5.1 GPIO e Gerenciamento de Portas

Os registradores **TRIS** (Tri-State) e **PORT** são fundamentais para o controle de I/O nos PICs:

- **TRISx (Data Direction Register)**:
  - Determina se os pinos são entradas (1) ou saídas (0)
- **PORTx (Data Latch Register)**:
  - Lê ou Escreve o estado lógico dos pinos se dado pino(s) começão ligados ou desligados
  - Podemos também com o comando `PORTxbits.RxN` específicar o pino que estamos utilizando

Boas Práticas

Ao contrário de algumas abstrações do Arduino, os Microcontroladores permitem manipulação direta de registradores usando diferentes bases numéricas podendo o programador escolher a que deseja utilizar entre decimal, binário ou hexadecimal.
Tanto o `TRIS` quanto o `PORT` podem ser definidos usando binários(exemplo: = 0b0), decimais(exemplo = 0) e hexadecimais(0x00) cabe então você escolher o melhor para dado uso, por exemplo, para definir um conjunto de `TRIS` diferentes podemos usar binário e definir então se são entradas ou saídas (0b01001011)
`OBS:` A ordem de leitura quando se define um conjunto é no formato decrescente e sempre configure os registradores `TRIS` antes de usar `PORT` para evitar estados flutuantes nos pinos.

5.1.1 Escrita do Código

Vamos então abaixo escrever um código simples que irá piscar um **LED** em uma quantidade x de tempo.

Listing 5.1: Programa para piscar LED no PIC16F628A

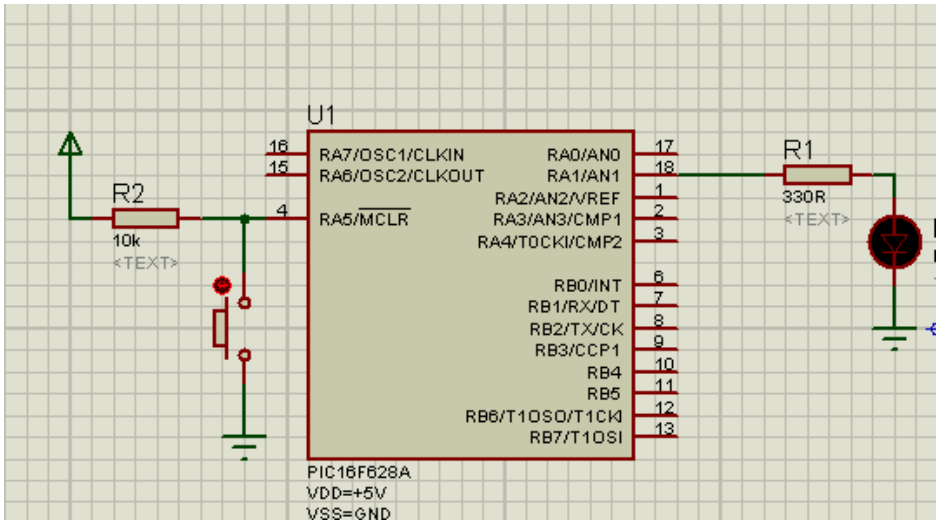
```c
/*
 * File:   main.c
 * Author: gabriel
 */

// CONFIG
#pragma config FOSC = INTOSCIO  // Oscillator interno
#pragma config WDTE = OFF       // Watchdog desativado
#pragma config PWRTE = OFF      // Power-up Timer desativado
#pragma config MCLRE = ON       // MCLR ativo
#pragma config BOREN = OFF      // Brown-out Reset OFF
#pragma config LVP = OFF        // Low Voltage Prog. OFF
#pragma config CPD = OFF        // Protecao de dados OFF
#pragma config CP = OFF         // Protecao de codigo OFF

#include <xc.h>

#define _XTAL_FREQ 4000000      // Frequencia de 4MHz
#define LED PORTAbits.RA1       // LED conectado em RA1

void main(void) {
    TRISA = 0;            // Todos os pinos PORTA como saida
    PORTA = 0x00;         // Inicializa PORTA em LOW

    while(1) {
        LED = 1;                // Liga LED
        __delay_ms(500);        // Espera 500ms
        LED = 0;                // Desliga LED
        __delay_ms(500);        // Espera 500ms
    }
}
```

5.1.2 Montagem de Circuito

Nessa seção trataremos da montagem de circuito, com base no código anterior precisaremos tente montar o circuito antes de olhar o diagrama dos seguintes componentes:

- **Componentes necessários**:
  - PIC16F628A
  - LED (qualquer cor)
  - Resistor (para o LED)
  - Resistor (para o MCLR)
  - Botão (para reset)
  - Fonte de alimentação $5V$
- **Esquemático**:
  - Conecte o LED entre RA1 e GND com resistor
  - Alimentação: V_{DD} (pino 14) em $5V$, V_{SS} (pino 5) em GND
  - MCLR (pino 4): Resistor para V_{DD} e botão para GND
  - Oscilador: Nenhum componente externo necessário (configuração INTOSCIO)

Abaixo está o circuito montado no simulador **Proteus**:



Importante: É sempre necessário ter o `MCLR` montado corretamente, ele tem a mesma função que o botão reset do Arduino.

5.2 Gravação do PIC

5.2.1 Ferramentas Necessárias

- **PICkit (Gravador)**: Dispositivo físico para comunicação entre computador e microcontrolador. Modelos recomendados:
  - PICkit 3 (suporte até PIC32)
  - PICkit 5 (modelo mais avançado até o momento)
- **Gravador USB K150:** Uma alternativa ao PICkit para mais informações acesse, www.makerhero.com/blog/como-utilizar-gravador-de-pic-usb-k150/
- **Soquete ZIF**: Facilita a gravação do PIC

5.2.2 Processo de Gravação com Soquete ZIF

1. Conectar o PICkit ao computador via USB
2. Inserir o microcontrolador no soquete ZIF:
   - Baixar alavanca para destravar o Soquete antes de inserir
   - A quantidade de pinos do PIC vai definir a sua posição no soquete, no PIC terá um círculo no canto superior esquerdo que indica a posição correta a ser colocada no soquete sempre para cima
   - Coloque o PIC sempre entre as linhas brancas que estão na parte de trás do soquete
   - Levantar alavanca para travar após posicionamento do PIC
   - Vire o soquete e perceba que nele terá marcações **J1, J2 e J3** que demonstra a posição que está o PIC e na parte de baixo dele o **DIP** que indicam a quantidade de pinos do μC e mais abaixo dele tem um conjunto de letras **J** que serão as configurações.
   - Volte para frente e coloque as configurações de **J** inferior vistas atrás do ZIF no **J** da parte superior respectiva
3. Ligue o PICkit ao ZIF observe atras e conecte corretamente, a seta que aponta para baixo no PICkit indica o **MCLR(Masterclear)**
4. Executar gravação na tela da IDE na opção **Make and program Device Main Project**
5. **Em caso de ERRO de VDD na hora da gravação:**
   - Na IDE clique no canto superior esquerdo em **File** → **Project Properties**
   - No menu que abrir na lateral esquerda vá em **PICkit** → **Option categories**, selecione **Power**
   - Nas opções de **Power** marque a primeira caixa e na opção de **Voltage**

Level coloque **4.625**

- Aplique as configurações e tente gravar novamente

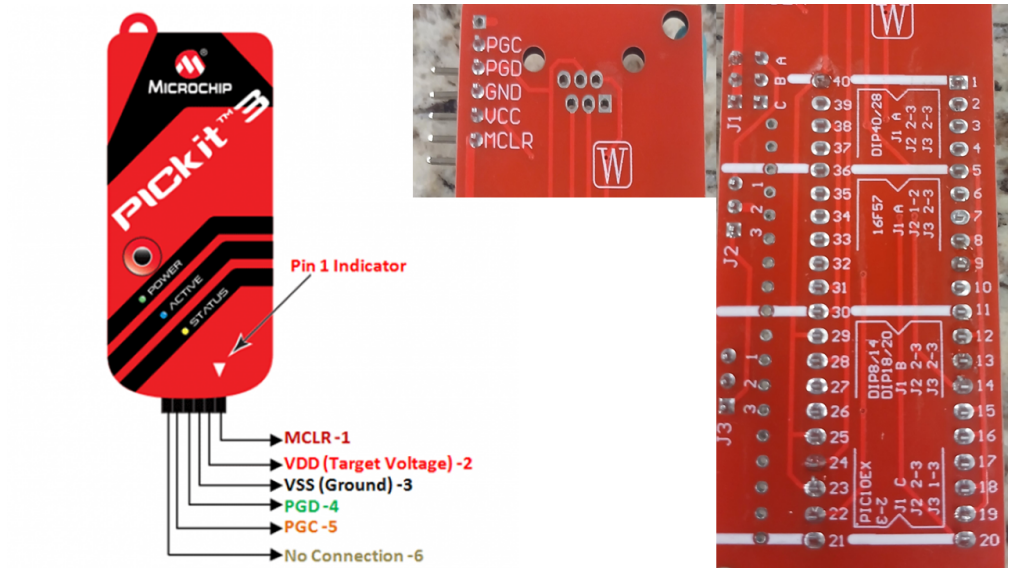


Figura 5.1: Conexão PICkit 3 com soquete ZIF

Após tudo isso o circuito do **PISCA LED** já deve estar funcionando e o **LED** piscando a meio segundo ou 500 ms.

ATENÇÃO

- Nunca force o PIC no soquete ZIF, danifica os pinos
- Se o PIC já estiver soldado na placa ainda podemos gravar novamente, contanto que não esteja travado a regravação, sem o uso do ZIF utilizando as portas equivalentes diretamente no PIC

5.3 Montagem do Cristal

Ao invés de se utilizar o **Oscilador Interno** do PIC, podemos montar um circuito com **Cristal** que é uma fonte de frequência mais precisa do que o interno do PIC, para a montagem podemos olhar no **datasheet**, pois lá se tem as informações todas as informações que se necessita até mesmo a montagem do Cristal.

Vamos precisar dos seguintes componentes:

- **Capacitores:** Dois capacitores, cada μC tem seu próprio valor mas os principais são:
  - **22 F ou 22 (real)**
  - **33 F ou 33 (real)**
- **Cristal:** Um cristal de qualquer frequência desde que o especifique no código do programa

Passos para a Montagem:

1. Conecte cada capacitor a um terminal do **Cristal**
2. Conecte a outro extremidade do cristal as portas **CLKIN** e **CLKOUT** do microcontrolador
3. Conecte a outra extremidade dos capacitores ao **GND**
4. Especifique no código a frequência do cristal

Abaixo esta o diagrama da montagem:

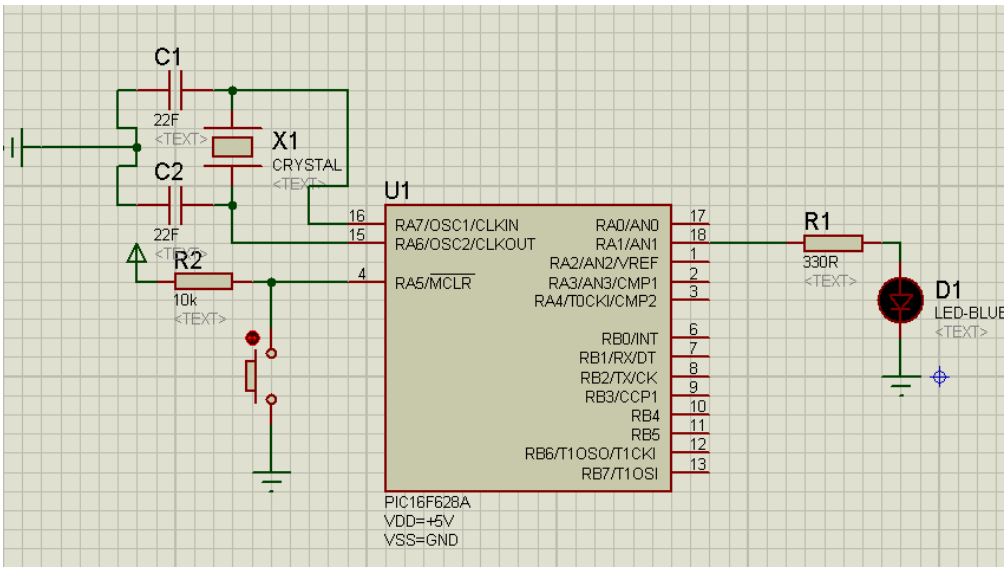


Figura 5.2: Montagem do Cristal

Capítulo 6

Módulos do PIC16

6.1 Timers e Interrupções

6.1.1 Timers

Os **Timers** são módulos internos do microcontrolador que contam **Pulsos de Clock ou Eventos Externos**, divididos em contadores que contam algum evento e temporizadores que contam intervalos de tempo, ambos não precisam bloquear a execução do programa diferente do comando `__delay_ms()`, os Timers são amplamente utilizado em sistemas em tempo real, ao chegar ao seu valor máximo ou a um valor pré-determinado assim como em **Displays** ele estoura (**Overflow**) e volta a contagem para zero, podendo nesse momento de **Estouro** acionar uma interrupção ou mecanismo. Em geral tirando alguns modelos em específico os **PICs** de 8-bits padrão possuem a mesma quantidade de timers, são eles:

- **Timer0:** É um timer de 8 bits simples, amplamente usado para delays curtos, eventos simples ou interrupções de baixa prioridade tendo somente o **Prescaler**.
  - Pode ser configurado usando o comando **TMR0** que conta de 0 a 255
  - Pode ser incrementado por fontes internas usando o **Clock Interno** com cálculo do $FOSC/4$ ou incrementado por fontes externas dependendo da configuração do $OPTION_REG$
- **Timer1:** É timer de 16 bits usado para contagens maiores e intervalos de tempo mais longos ou maior precisão
  - Divido em 2 registradores que formam 16 bits com comandos **TMR1H** e **TMR1L** que contam até 65.535
  - Pode funcionar em modo **Timer** que incrementa com o clock interno ou em modo **Count** que conta pulsos externos
  - Pode ser configurado com o comando **T1CON**

- **Timer2:** É um timer de 8 bits com recursos de **prescaler**, divide a frequência antes do clock interno, e **postscaler**, divide a frequência do evento/interrupção, utilizados para gerar uma contagem mais precisa a partir de um valor definido ou aplicações com **PWM**
  - Pode ser utilizado com o comando **TMR2** que vai fazer a contagem e o comando **PR2** que define o período
  - Quando **TMR2** atinge o valor definido em **PR2**, ele reinicia e pode acionar uma interrupção.

6.1.2 Configuração de Prescaler/Postscaler

PS2	PS1	PS0	Taxa
0	0	0	1:2
0	0	1	1:4
0	1	0	1:8
0	1	1	1:16
1	0	0	1:32
1	0	1	1:64
1	1	0	1:128
1	1	1	1:256

Tabela 6.1: Prescaler do Timer0 (OPTION_REG)

Prescaler	Taxa	Postscaler	Taxa
00	1:1	0000	1:1
01	1:4	0001	1:2
1x	1:16	0010	1:3
...	...	...	...
...	...	1111	1:16

Tabela 6.2: Prescaler e Postscaler do Timer2 (T2CON)

6.1.3 Interrupções

Interrupções são mecanismos que permitem que o μC suspenda a execução de programa para executar uma rotina de tratamento com o comando **(ISR – Interrupt Service Routine)** em resposta a um evento específico. Essa abordagem possibilita a resposta imediata a eventos externos ou internos, sem a necessidade de **polling constante (verifica repetidamente se houve uma mudança em**

um evento), microcontroladores **PIC** de 8-bits não possuem prioridade de interrupções nativa ao contrário de modelos 16/32 bits.

Tipos de interrupções:

- **Interrupção Gobal (GIE):** Para que qualquer interrupção seja processada, o bit global de interrupções (GIE) e, geralmente, o bit de interrupções periféricas (PEIE) devem ser configurados para 1.
- **Interrupção Específica (Cada Módulo):** Timer0, Timer1, Timer2, comunicação serial, entrada externa e entre outros módulos possuem seu próprio comando de **Interrupt Enable Bit** e de **Flag**, por exemplo:
  - **Timer0 (T0IE)** e se a flag foi configurada por **T0IF** ele executa a ISR
  - **Timer1 (TMR1IE)**e se a flag foi configurada por **TMR1F** ele executa a ISR
  - **Timer 2(TMR2IE)**e se a flag foi configurada por **TMR2IF** ele executa a ISR .

Exemplo de Código: Gerar uma interrupção e alternar o estado de um LED conectado a um pino

Listing 6.1: Programa para relogio e LCD no PIC16F628A

```
/*
 * File:   main.c
 * Author: gduar
 *
 * Created on 3 de Abril de 2025, 11:02
 */

// CONFIG
#pragma config FOSC = XT // Oscillator Selection bits (INTOSC
    oscillator: I/O function on RA6/OSC2/CLKOUT pin, I/O
   function on RA7/OSC1/CLKIN)
#pragma config WDTE = OFF       // Watchdog Timer Enable bit
   (WDT disabled)
#pragma config PWRTE = OFF      // Power-up Timer Enable bit
   (PWRT disabled)
#pragma config MCLRE = ON       // RA5/MCLR/VPP Pin Function
   Select bit (RA5/MCLR/VPP pin function is MCLR)
#pragma config BOREN = OFF      // Brown-out Detect Enable
   bit (BOD disabled)
#pragma config LVP = OFF        // Low-Voltage Programming
   Enable bit (RB4/PGM pin has digital I/O function, HV on
   MCLR must be used for programming)
```

```
#pragma config CPD = OFF        // Data EE Memory Code
   Protection bit (Data memory code protection off)
#pragma config CP = OFF         // Flash Program Memory Code
   Protection bit (Code protection off)

#include <xc.h>

#define _XTAL_FREQ 4000000
#define LED PORTBbits.RB0

volatile uint8_t temporizador = 0;

volatile struct {
    volatile uint8_t SEG;
    volatile uint8_t MIN;
} relogio = {0, 0};


void main(void) {
    TRISB = 0x00;
    PORTB = 0x00;

    iniciar_timer0();

    while (1){
    LCD_SetCursor(0,0);
    LCD_Print(relogio.SEG);
    LCD.SetCursor(0,1);
    LCD.Print(relogio.MIN);
    }
}

void __interrupt() isr() {
    if (INTCONbits.T0IF) {
        temporizador++;
        if(temporizador >= 15){
            relogio.SEG++;
            temporizador = 0;
        }
        if(relogio.SEG >= 60) {
            relogio.SEG = 0;
            relogio.MIN++;
        }
```

```
        TMR0 = 0;              // Ajusta para overflow a cada
           ~65.5ms
        INTCONbits.T0IF = 0;   // Limpa flag de interrupção
    }
}


void iniciar_timer0() {
    OPTION_REG = 0b00000111; // Prescaler 1:256, clock
       interno
    TMR0 = 0;                // Inicia contagem em 0
    INTCONbits.T0IE = 1;     // Habilita interrupção do
       Timer0
    INTCONbits.GIE = 1;      // Habilita interrupções globais
}
```

6.2 Comparadores

Os **Comparadores Analógicos** são periféricos integrados em **PICs** que permitem comparar dois sinais analógicos e gerar uma saída digital indicando qual sinal é maior. Eles são utilizados em aplicações que envolvem sensores analógicos, controle de tensão e conversões analógico-digitais simplificadas. O **PIC16F628A** contém 2 comparadores que estão presentes nos pinos **RA0 a RA3** que pode ser configurado pelo comando **CMCON (Comparator Module Control Register)**.

Os bits dele podem ser configurados por:

- **CM2 e CM0:** Seleciona a configuração dos comparadores
  - Ambos os comparadores habilitados
  - Ambos desabilitados
  - Um comparador habilitado com referência interna (**VREF**)
- **CIS:** Alternas as entradas positivas e negativas
- **C1OUT e C2OUT:** Indicador do estado de saída dos comparadores
- **C1INV / C2INV:** Inverte a polaridade da saída

Modos de Operação:

- **Independente:** Cada comparador opera de forma separada com entradas diferentes para V^+ e V^-

CM2:CM0	Hex	Modo de Operação
000	0x00	Comparadores independentes
001	0x01	Três entradas multiplexadas
010	0x02	Comparador com referência interna
100	0x04	Duas entradas comuns (invertidas)
111	0x07	Comparadores desativados (pinos como I/O digitais)

Tabela 6.3: Modos de Operação dos Comparadores (PIC16F628A)

- **Referência Comum (VREF):** Ambos os comparadores compartilham a mesma entrada positiva V^+ e independentes V^-

6.3 Comunicação Serial (USART)

Capítulo 7

Programação Avançada de PIC

7.1 Option Registers

7.2 Status

7.3 Bit Baggin

7.4 Debounce

7.5 Otimização

Nessa seção trataremos de uma programação

Capítulo 8

Periféricos Essenciais

8.1 LCDs

8.2 Conversor A/D

Capítulo 9

Projetos Baseados em Problemas

9.0.1 Sistema de Controle

9.0.2 Monitor de Temperatura

9.0.3 Interface Homem-Máquina